

On the use of exponential basis functions in the PINN architecture: An enhanced solution approach for the Laplace, Helmholtz, and elasto-static equations

Sadegh Ghamsari Esfahani, Bashir Movahedian^{*}, Saeid Sarrami, Mojtaba Azhari

Department of Civil Engineering, Isfahan University of Technology, Isfahan 84156-83111, Iran

ARTICLE INFO

Keywords:

Physics-informed neural network
Exponential basis functions
Laplace and helmholtz equations
Linear and nonlinear elasto-static problems
Inverse material property identification
Complex valued solutions

ABSTRACT

This paper presents a novel physics-informed neural network (PINN) architecture that employs exponential basis functions (EBFs) to solve many boundary value problems. The EBFs are organized so that the PINN architecture may employ their simple differentiation features to solve partial differential equations (PDEs). The proposed approach has been meticulously investigated and compared to the conventional PINN in the solution of many instances involving Laplace and Helmholtz differential equations, as well as linear and nonlinear elasto-static problems. By utilizing EBFs, fewer interpolation functions are needed and the derivation rule can be explicitly applied, significantly reducing computation time. In addition, in some problems, PINN embedded with EBFs shows better precision near the boundaries of the problem, which is one of the disadvantages of conventional PINNs. Two specific applications of the presented method have been investigated, namely the inverse identification of material properties and the solution of complex-valued partial differential equations. The proposed method was found to be accurate and converging when applied to inverse problems. As shown, it is also suitable for the solution of PDEs containing both imaginary and real parts.

1. Introduction

Traditional methods, while valuable, often come with inherent challenges that hinder their efficacy in providing easily attainable solutions. As we strive for more robust and versatile methodologies, the intersection of physics and artificial intelligence has paved the way for innovative solutions. For the past few years, using Artificial Neural Networks (ANN) has drawn attention in different fields, such as image classification, handwriting and speech recognition, and translation [1,2]. ANNs also commonly known as Deep Neural Networks (DNNs) have advanced and progressed in many areas, including geoscience [3], material sciences [4–6], health monitoring [7,8], etc. ANN has also found its way through many engineering fields, from experimental to analytical. A new class of ANN, known as Physics-Informed Neural Networks (PINN), has been developed recently [9–14]. In these networks, a partial differential equation (PDE) with boundary conditions is implemented in the layers and losses. By providing nodes in the domain and boundaries as inputs, a specific set of loss functions is defined and minimized to obtain a converged solution. In this approach, differentiations are carried out using graph-based computations integrated into TensorFlow as a specific instance of automatic differentiation (AD).

TensorFlow is an interface for expressing machine learning algorithms and an implementation for executing such algorithms [15].

^{*} Corresponding author.

E-mail address: b.movahedian@iut.ac.ir (B. Movahedian).

A computation expressed using TensorFlow can be executed with little or no change on a wide variety of heterogeneous systems, ranging from mobile devices such as phones and tablets up to large-scale distributed systems of hundreds of machines and thousands of computational devices such as GPU cards.

Unlike conventional techniques that rely on predefined equations, PINN leverages the capacity of neural networks to learn and adapt, embedding physical laws directly into their architecture. This not only circumvents the limitations of traditional methods but also opens new avenues for addressing complex structural dynamics. Building upon the foundation laid by PINN, the Scientific Neural Network (SciANN [16]) extends the capabilities of scientific machine learning, offering a versatile framework specifically designed for solving scientific problems [17]. Lu [18] introduced a deep learning library, DeepXDE [19], for solving differential equations. The library provides a framework for implementing PINN and deep learning techniques to address a wide range of problems involving differential equations.

Several groundbreaking works have demonstrated the efficacy of PINN and SciANN in structural analysis. Raissi et al. introduced the concept of PINN to solve forward and inverse problems associated with PDEs [9]. The authors demonstrated PINN's ability to learn underlying physical laws and solve complex PDEs using neural networks. The authors showed the application of PINN to predicting structural deformations, revealing its capacity to generalize across various structural forms. Raissi et al. focused on deep learning to discover hidden physics models, specifically nonlinear partial differential equations [20]. The authors proposed a framework to learn the underlying governing equations directly from data, showcasing the potential of deep learning in uncovering hidden physical laws. A physics informed deep learning framework capable of encoding a vital class of physical laws governing fluid motions, namely the Navier-Stokes equations was presented by M. Raissi, A. Yazdani, and G. E. Karniadakis [21]. The study sought to leverage the conservation laws (i.e., mass, momentum, and energy) to infer hidden quantities of interest, such as velocity and pressure fields merely from spatio-temporal visualizations of a passive scalar (e.g., dye or smoke), transported in arbitrarily complex domains (e.g., in human arteries or brain aneurysms). Results demonstrated that the methodology can be used in physical and biomedical problems to extract valuable quantitative information. Using SciANN, Kianian et al. [22] performed bending analysis of nanobeams on a three-parameter nonlinear elastic foundation including hardening and softening effects using nonlocal elasticity theory. The paper investigated both forward and inverse problems. Another work by Haghighat et al. was performed using SciANN for inversion and surrogate modeling in solid mechanics [23]. They used PINN implemented with the SciANN library to perform inversion and surrogate modeling in solid mechanics. In this study, the momentum balance and constitutive relations were incorporated into PINN. Also, a multi-network model that results in a more accurate representation of field variables was proposed. Findings indicated that the PINN models can accurately predict the solution to a wide range of parameters newly introduced to the network. Moreover, the authors developed a deep learning model to solve direct and inverse problems in structural mechanics and vibrations [24]. Another work for inverse modeling of non-isothermal multiphase poromechanics using physics-informed neural networks was carried out in [25] by Amini et al. in which a solution strategy for parameter identification in multiphase thermo-hydro-mechanical (THM) processes in porous media using PINN was introduced.

Liu Yang et al. in [26] focus on using physics-informed generative adversarial networks (PI-GANs) to solve forward, inverse, and mixed stochastic problems in a unified manner based on a limited number of scattered measurements. They encoded the architecture of GAN and the governing physical laws in stochastic differential equations (SDEs). The results showed the accuracy and effectiveness of PI-GAN in solving SDEs in up to 30 dimensions. As a result, PI-GAN could tackle very high dimensional problems given more sensor data with low-polynomial computational cost growth. Furthermore, Lusch et al. explored the use of deep learning for universal linear embeddings of nonlinear dynamics [27]. This study was conducted to identify coordinate transformations that make strongly nonlinear dynamics approximately linear, which has the potential to enable nonlinear prediction, estimation, and control using linear theory. They used this method to study plane-parallel flows, which are common in fluid dynamics.

On this subject, Chang et al. [28] introduce a novel approach using semi-analytic PINN called singular layer PINN (or sl-PINN). Liu et al. [29] used variable separated physics informed neural networks based on adaptive weighted loss functions (AW-vsPINN) for blood flow model in arteries. They proposed sub-neural networks to separately predict the unknown scalar state variables by sharing the same input layer. Another work carried out by S. Tang et al. was focused on utilizing PINN combined with interpolation polynomials (PI-PINN) to solve nonlinear partial differential equations [30]. In their work, it has been shown that PI-PINN structure is effective in solving nonlinear partial differential equations. A coupled physics-informed neural network (CPINN) was presented by W. Zhang and J. Li [31] for the closed-loop geothermal system. The advantage of this method is described that it avoids grid generation compared with the grid-based methods and it is parallel for the equations and variables. To gather multiple solutions of nonlinear elliptic differential equations, Yao Huang et al. [32] proposed homotopy physics-informed neural networks, which is implemented by composition of the training of a fully connected neural network, named the starting neural network, and training processes of several PINNs with different tracking parameters. Also, Salvatore Cuomo et al. [33] proposed a physics-informed learning approach to Bernoulli-type free boundary problems. They worked on a variational formulation for Bernoulli free boundary problems embedded in PINN. An approach based on quasi Monte-Carlo integration methods has been implemented for the calculation of loss functions containing integral terms. Another work carried out by Difonzo et al. concentrated on solving inverse problems in peridynamic models [34]. They used Radial Basis Functions (RBF) in PINN. RBFs are used as the first layer activation functions. The authors have used peridynamic formulation embedded in PINN to solve integro-differential equations.

PINNs provide a powerful framework for solving PDEs and other mathematical models. However, they have several limitations. One major challenge is their strong dependence on network architecture and hyperparameters, such as the number of layers, neurons, and learning rates, which significantly impact convergence and accuracy. Additionally, training PINNs is computationally expensive, especially for complex problems or high-dimensional domains. Another notable limitation is the difficulty of accurately enforcing boundary and initial conditions, particularly for stiff or discontinuous problems, where standard neural networks may struggle to

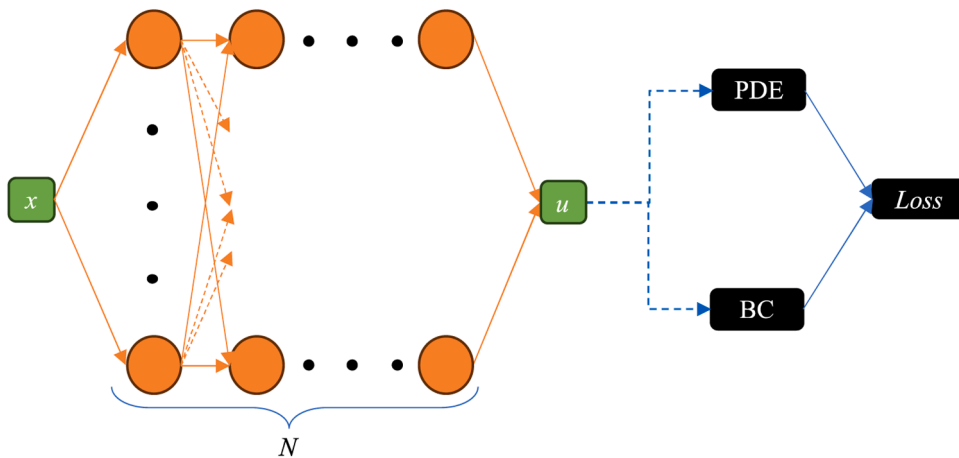


Fig. 1. The schematic view of the PINN network for general boundary value problems.

satisfy constraints without overfitting or introducing numerical errors, as discussed in [35]. Furthermore, PINNs suffer from loss imbalance, where certain terms in the loss function dominate, leading to suboptimal optimization and convergence. While PINNs effectively approximate smooth solutions, they often face difficulties in capturing non-smooth or discontinuous solutions due to the inherent limitations of traditional neural network architectures. These challenges highlight the need for further research and improvements to enhance the robustness and applicability of PINNs.

In 2010, for the first time, the idea of using exponential basis functions (EBFs) that satisfy the governing differential equation was introduced [36]. In this research, two approaches of optional selection of complex-valued exponents are presented for the EBFs. Finally, the answer to the governing equation was expressed as a linear combination of EBFs. In the following, the application of the proposed method in the solution of various problems such as elasto-dynamic wave propagation, and inverse and direct heat conduction problems were investigated. Since choosing bases that satisfy the strong form of the equation is only possible for linear equations with constant coefficients, the method has limitations in solving equations with non-constant coefficients and nonlinear differential equations. Shamsaei and Boroomand used EBF for the solution of laminated structures in 2011 [37]. The authors utilized exponential basis functions to solve the governing equations of laminated structures, ensuring compatibility and equilibrium at the interfaces. Their method demonstrated excellent agreement between the obtained results and some benchmark problems. Zandi et al. [38] utilized EBFs in the solution of incompressible fluid problems with moving free surfaces. Utilizing a Lagrangian formulation, their approach significantly improved the accuracy and efficiency in addressing these complex fluid dynamics problems.

This paper aims to build upon the advancements made by these pioneering studies, delving into applying PINN to some boundary value problems (BVPs) with a particular emphasis on 2D scenarios. As a result of combining insights from our previous research [36, 39–41], we present an innovative form of PINN that employs an exponential basis as activation functions into neural networks without hidden layers. Unlike previous studies on the implementation of EBFs in which the coefficients of the exponents were chosen so that the strong form of the equation could be satisfied, in this work the exponents are optimized along with the other variables so that the boundary and domain equations are satisfied simultaneously. Thus, it became possible to solve nonlinear PDEs, which was not applicable before [36]. For more details on the implementation of the presented method, a sample code can be downloaded through the following links: <https://codeocean.com/capsule/6754754/tree/v1> [42] or <https://github.com/sghamsari/EBF-PINN-Laplace-Solution/>.

The outline of this paper is as follows. In Section 2, the methodology and the formulations of regular PINNs and EBF-PINNs are introduced. Section 3 is dedicated to solving five sample boundary value problems. First, the Helmholtz problem is investigated and the EBF-PINN results are compared to the conventional PINN. The accuracy of the presented method near boundaries is the main target of this example. The second example considers the Laplace equation. This example shows the method's capability to include imaginary solutions. The effect of the number of EBFs is another issue discussed in this example. In the third example, an elasto-static plane-strain problem is solved using the EBF-PINN and regular PINN. The precision of the results and convergence speed are compared for both methods. After that, a linear and nonlinear elasto-static elasticity problem with severe nodal boundaries is investigated and the results are compared to the exact solution. The last example focuses on parameter identification in the elasticity inverse problem. In this example, the convergence of parameter identification in both regular PINN and EBF-PINN is observed and compared. A fast convergence concerning time and epoch's number is shown in that example. A summary of the results obtained is presented in Section 4, as well as future directions.

2. Methodology

In this chapter the methodology and the formulation of regular PINNs and EBF-PINN are presented and compared.

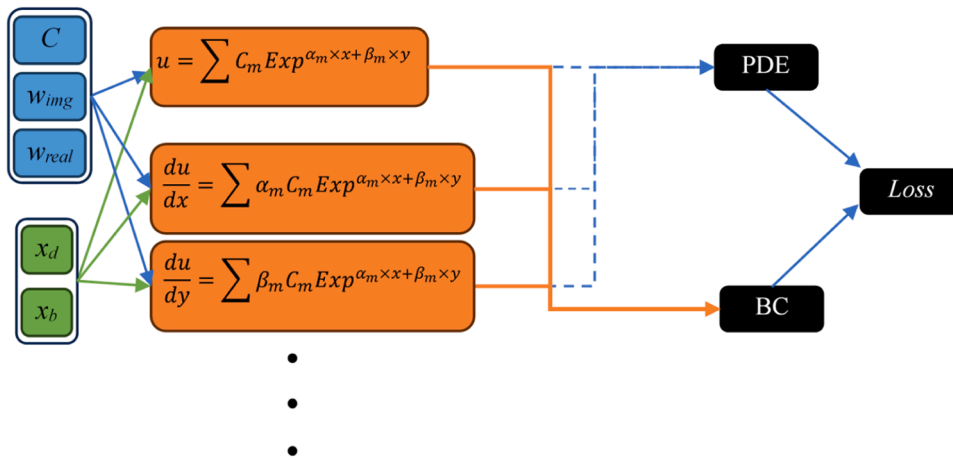


Fig. 2. Schematic network designed to be used in the PINN embedded by EBFs.

2.1. Introduction to PINN

An overview of PINN architecture is presented in this section. Considering an l -layer network with the input vector x , the output vector u , and variable parameters W and b , each layer creates data for the next layer by the following nested transformation,

$$z^l = \sigma^l(W^l z^{l-1} + b^l), \quad l = 1, 2, \dots, N \quad (1)$$

W is called weight and b is known as bias in neural networks. N is the number of layers and σ is the activation function of each layer. σ is commonly considered as a tanh function for the middle layers and a linear function for the last layer. The inputs of the first layer, i.e. z^0 , is x (the nodal coordinates) and the outputs of the last layer, i.e. z^N , is u (the solution of the governing equation). The last layer results are used in PDEs and boundary condition equations to create the loss function. This pattern represents the conventional form of PINN with a deep learning scheme that is used in conventional PINNs like those developed in Refs [9,12,21] and [23–25]. Fig. 1 demonstrates the graph configuration of PINN.

The number of layers (N) and neurons is chosen depending on the governing PDE complexity and boundary conditions. To optimize the model's weights and minimize loss, various algorithms have been proposed. Among them, Adam [43] and Stochastic Gradient Descent (SGD) [44] are among the most efficient and widely used. These algorithms operate based on the model's gradient with respect to its trainable parameters, which typically include all weights (W) and biases (b). In the process of guiding the model toward minimal loss, certain additional parameters, known as hyperparameters, play a crucial role.

One of the important parameters, which should be selected carefully to get desire loss during the training of the model, is called learning rate. This parameter can be simply introduced as the rate of changing the trainable parameters during the training. As the learning rate gets higher, optimization steps become wider and vice versa. Although using large steps may lead to faster training, it might cause instability near the local minima. On the other hand, while using small learning rate leads to more accurate results; it slows down the training procedure. A linear or exponential change of learning rate is used to achieve the best results [45].

Another crucial parameter in training a model is the batch size, which refers to the number of data samples fed into the model in each iteration to optimize the variables. It is important to note that models are typically trained on large datasets. Using a single data point for optimization at each step would be highly inefficient and computationally expensive. Therefore, the dataset is randomly divided into smaller batches, which are sequentially fed into the model during training. An epoch refers to a complete pass of the entire dataset through the model during the optimization process. A model may require hundreds or even thousands of epochs to achieve optimal performance. In general, batch size, learning rate, and the number of epochs are considered hyperparameters. To ensure model convergence, these hyperparameters must be carefully selected, often through a process of trial and error.

Some studies, like [46] suggest that while the Adam algorithm increases convergence speed, it may result in lower accuracy compared to SGD. However, Choi et al. [47] demonstrated that improved optimization algorithms like Adam can achieve better performance, if the hyperparameters are properly tuned.

Another important term in PINN is patience. As mentioned earlier, using large learning rate can cause instability during the later stages of training. Patience refers to a condition where the learning rate is reduced if no improvement is observed after a certain period.

It is worth mentioning that although increasing the batch size can accelerate the training process, it may result in lower accuracy as the error is distributed over more points. On the other hand, an excessively small batch size can lead to significant fluctuations in the error reduction process. Another important set of hyperparameters in defining and training the model is the number of hidden layers and the number of neurons in each hidden layer. Increasing these parameters enhances the complexity of the model, which can lead to greater flexibility. However, it may also make convergence more challenging due to the potential overparameterization of the model.

2.2. Introduction to EBF-PINN

To implement EBFs in neural network architecture, without losing the generality of the method, we transform the layers for two-dimensional (2D) problems in the following form,

$$u(x, y) = \sum_{m=1}^{N_b} C_m e^{\alpha_m x + \beta_m y} \quad (2)$$

in which, x and y are the inputs, and N_b is the number of base functions used for expressing the solution. Also, $\alpha_m = w_{real,x,m} + i w_{img,x,m}$ and $\beta_m = w_{real,y,m} + i w_{img,y,m}$ where $i = \sqrt{-1}$. The real and imaginary components of α_m and β_m are represented by $w_{real,m}$ and $w_{img,m}$, respectively, and C_m is the weight of the m th EBF, which is also a complex number. All of the exponents and weights including $w_{real,x,m}$, $w_{img,x,m}$, $w_{real,y,m}$, $w_{img,y,m}$ and C_m are considered trainable in the network.

According to the Euler's relation, EBFs with complex exponents are an alternative representation of harmonic sine and cosine functions, which possess the orthogonality property and are well-suited for interpolation. Consequently, using EBFs with complex exponents enhances the robustness of the solution and enables it to effectively capture imaginary components of the solution.

Remark 1. There are two primary approaches for using Exponential Basis Functions (EBFs) to solve Partial Differential Equations (PDEs). The most well-known method combines EBFs with the Trefftz technique [36–41]. In this approach, the exponents of the EBF are configured so that the resulting basis exactly satisfies the strong form of the governing PDE. A set of nodes is then selected on the boundaries, and the constant coefficients of the solution series are computed to ensure that the boundary conditions are satisfied, typically using a point collocation method.

The second approach, which has not been explored until now, involves using a neural network to determine both the exponents and coefficients through an optimization process. This ensures that both the PDEs and boundary conditions are approximately satisfied via a collocation technique. In this paper, we utilize a general form of EBFs, where both exponents are independently trained through the optimization process. As a suggestion for further research, the first type of EBF, i.e., those derived using the Trefftz technique as outlined in [36–41], could be implemented to solve the problems discussed in those works. ■

In TensorFlow, however, it may require more complicated programming, particularly in defining the loss function. On the other hand, in the proposed formulation, the required derivatives of series solutions, e.g., $\partial u / \partial x$ or $\partial u / \partial y$, are calculated directly. This means that there is no need for graph-based differentiation calculations, as required in conventional PINNs. Unlike the results presented in the numerical examples section, this feature reduces computational costs. The network output is then assigned to represent functional fields, such as u and v , which correspond to the horizontal and vertical components of the displacement field in the elasticity problem.

The model is expanded to calculate a set of losses, which should be minimized. Therefore, governing PDE and boundary conditions are generated using graph-based computation, and a final loss is returned. Applying governing differential operator $\mathcal{L}$ on u incorporated in cost function $\mathcal{R}$ along with the residual of boundary conditions results in,

$$R = |u - \bar{u}|_{\Omega} + |\mathcal{L}u - \bar{b}|_{\Omega} + |u - \bar{u}|_{\Gamma}, \quad (3)$$

where Ω and Γ indicate the domain and boundary of the problem. The overbar sign represents known data and u is the unknown solution to be computed as the network output. Additionally, the norm in Eq. (3) can represent any appropriate method for calculating the error, such as the mean square error or mean absolute error. Fig. 2 illustrates the schematic structure of the designed network for the proposed method.

In Fig. 2, the blue boxes depict variables, which are considered trainable in the network, and the green ones represent inputs. Also, x_d and x_b represent the coordinates of domain and boundary points where the PDE and boundary conditions losses are calculated, respectively.

Other required derivatives can be calculated explicitly in the same way as shown. One of the benefits of this method is that the PDE and boundary conditions inputs are inserted into the same model through different graphs. With this technique, boundary conditions are imposed more easily than conventional methods. This results in faster convergence and run time of each iteration than in the case of conventional methods.

In conventional PINNs, such as those implemented in frameworks like SciANN, the nodes for boundary conditions and PDEs are combined into a single array in random order. To ensure that domain nodes and boundary nodes contribute only to their respective loss functions, i.e., PDE or boundary equations, additional sample weight arrays are incorporated into the model. For instance, if 1000 nodes are selected within the domain and 500 nodes are chosen on the boundaries, these are combined into a single input array of size 1500. This input array is paired with two sample weight arrays; the first array assigns a value of 1 to the domain nodes and 0 to the boundary nodes, while the second array assigns 0 to the domain nodes and 1 to the boundary nodes. This mechanism ensures proper assignment of contributions to the respective loss terms. For example, a batch of 50 nodes might randomly include 30 domain nodes and 20 boundary nodes, with their contributions determined by the associated sample weights.

In contrast, the present study introduces a more structured and efficient approach, referred to as parallel input insertion. Instead of merging all nodes into a single array, separate inputs are defined for domain nodes, x_d , and boundary nodes, x_b . Each input independently feeds into the model, producing outputs specifically for the corresponding equations. For example; an input corresponding to the domain computes outputs for the domain equations, while another input for the boundary computes outputs for the boundary

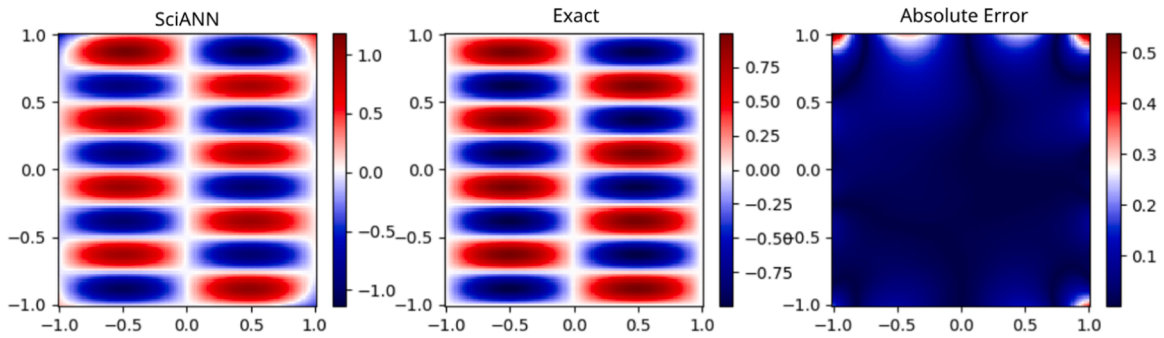


Fig. 3. Variations of the numerical solution obtained by SciANN (conventional PINN), exact solution and absolute error of the Helmholtz equation. (Model has been run with 4 layers, 40 neurons per layer, 20,000 domain and boundary nodes, and 1000 iterations).

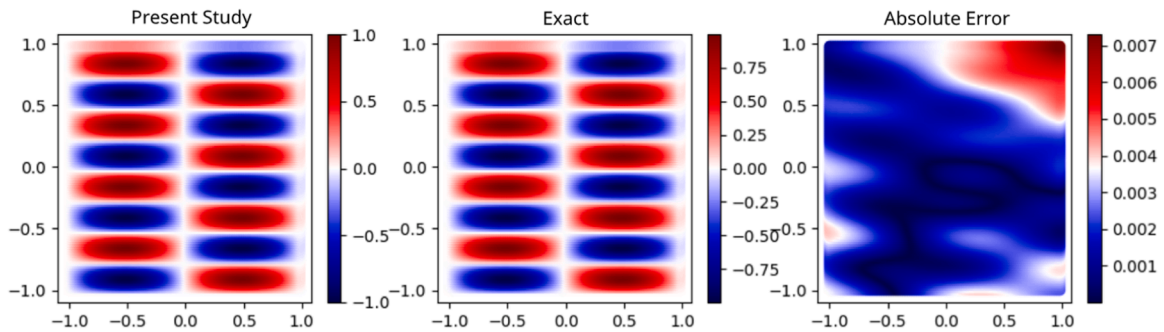


Fig. 4. Variations of the numerical solution obtained by EBF-PINN, exact solution and absolute error of Helmholtz equation. (Model has been run with 100 bases, 20,000 domain and boundary nodes, and 1000 iterations).

equations. These inputs are fed into the model simultaneously, and the respective losses for domain and boundary equations are calculated concurrently. This structured separation eliminates the need for additional sample weight arrays and allows for better utilization of computational resources. This parallel input insertion approach demonstrated significant improvements, particularly in execution time, while maintaining or enhancing the model's accuracy and efficiency. More details on the parallel feeding of domain and boundary nodes are provided in [Appendix A](#), where the solution to the sample problem of the Laplace equation is presented.

Different algorithms are used to train neural networks. Among the algorithms we tested, Adam optimization showed the best convergence and accuracy convergence and accuracy [43].

Several parameters influence the rate of convergence and the accuracy of results, including batch size, the number of epochs, and the learning rate, as previously mentioned. Notably, when employing EBFs in neural networks, the learning rate must be handled differently from traditional PINNs. Studies have shown that achieving high accuracy often requires a very low learning rate. Furthermore, using a small batch size in these networks has been observed to improve both convergence and accuracy while maintaining high computational efficiency.

Remark 2. One of the most significant features of PINN is its ability to solve inverse problems. To this end, the unknown parameters that need to be determined are incorporated into the neural network as trainable variables. To train the network, a set of random data on the domain is given as input data. By reshuffling them over epochs, trainable variables are optimized. The following section shows the performance of the presented method for different problems. It is evident that regularizing parameters mitigates the oscillation of loss through training [48]. However, our study showed that decreasing batch size leads to better convergence and accuracy. ■

3. Numerical examples

This section examines five sample problems using conventional PINN and EBF-PINN regarding convergence, accuracy, and computational time. The direct solution of the benchmark problem of the Helmholtz equation is investigated in the first example. The second example demonstrates the method's ability to handle imaginary solutions by looking at a Laplace equation containing real and imaginary parts. We discussed the forward solution of plane strain elasto-static problems in the third and fourth examples, while we examined the inverse identification of material properties in the last example.

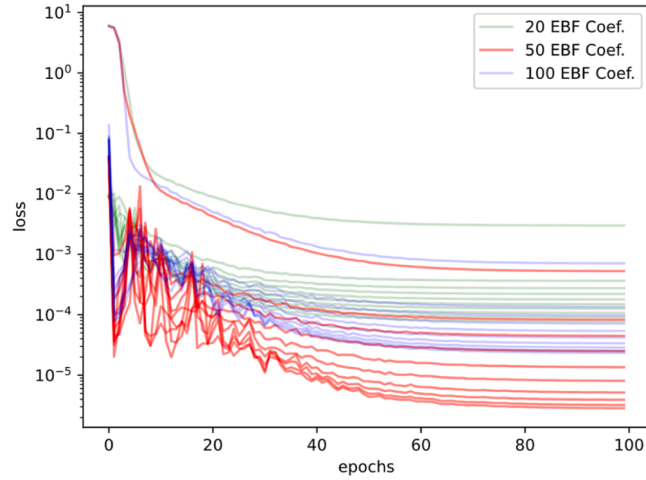


Fig. 5. Variations of EBF-PINN loss function vs number of epochs used in the solution of Laplace equation by employing 20, 50, and 100 EBFs.

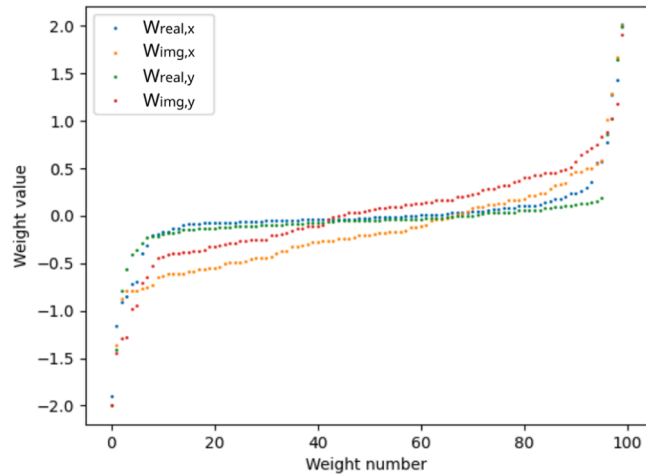


Fig. 6. Variations of the exponent's coefficients, $w_{real,x,m}$, $w_{img,x,m}$, $w_{real,y,m}$ and $w_{img,y,m}$, in case of using 100 EBFs obtained by training neural network formed for sample problem of Laplace equation.

3.1. Helmholtz forward example

Solving the Helmholtz equation plays a key role in most PINN researches. The equation is as follows,

$$\nabla^2 u(x,y) + k^2 u(x,y) = q(x,y), \quad (x,y) \in \Omega = (-1, 1)^2 \quad (4)$$

$$u(x,y) = h(x,y), \quad (x,y) \in \Gamma \quad (5)$$

where ∇^2 is the Laplace operator. An exact solution to this problem can be assumed as $u(x,y) = \sin(a_1 \pi x) \sin(a_2 \pi y)$. The $q(x,y)$ for this assumption equals to,

$$q(x,y) = -(a_1 \pi)^2 \sin(a_1 \pi x) \sin(a_2 \pi y) - (a_2 \pi)^2 \sin(a_1 \pi x) \sin(a_2 \pi y) + k^2 \sin(a_1 \pi x) \sin(a_2 \pi y) \quad (6)$$

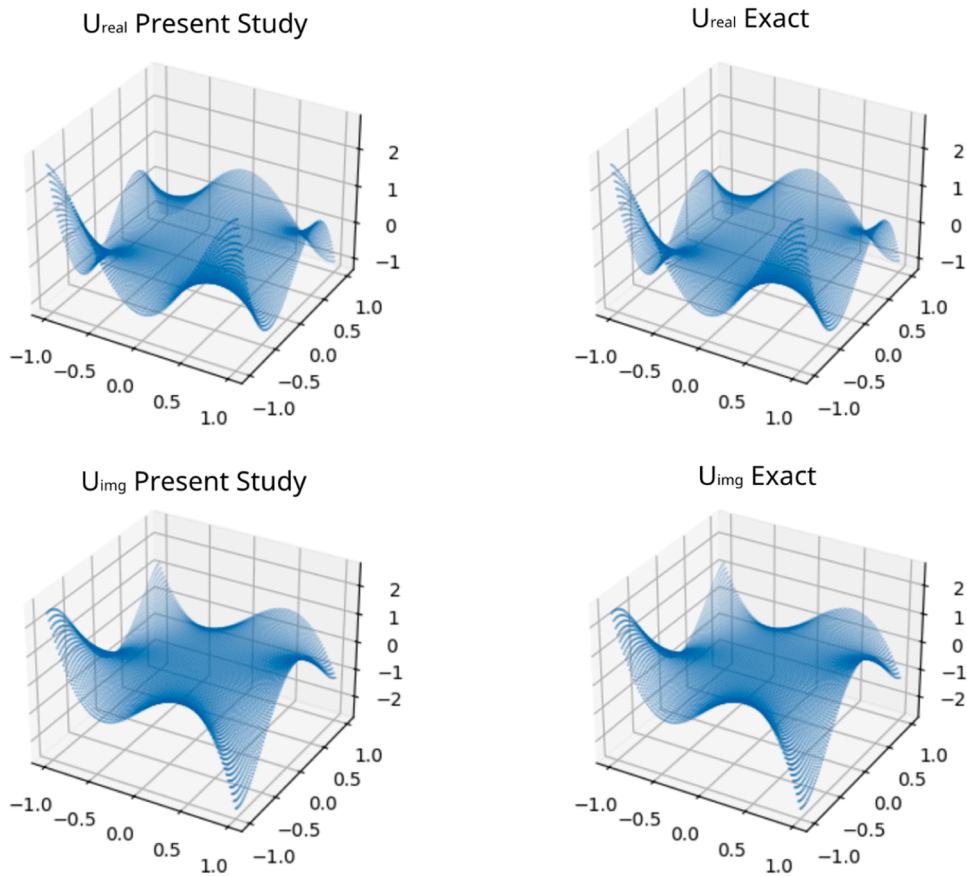


Fig. 7. Variations of real and imaginary parts of the solution obtained by EBF-PINN with 20 EBFs and exact solution.

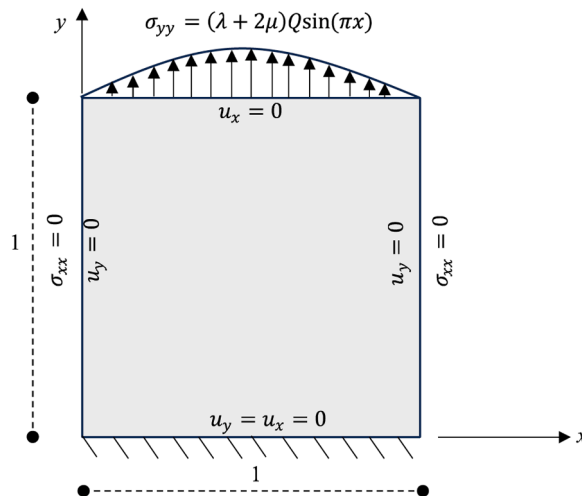


Fig. 8. The geometry and boundary conditions of the elasto-static plane-strain problem investigated in Section 3.3.

In this example, previously introduced in [49], a_1 and a_2 are taken as 1 and 4, respectively. Also, the wave number, k , is 1. The PINN approximation with a deep neural network is constructed using SciANN with 4 layers and 40 nodes for each layer. These nodes are fully connected. Tanh is used as the activation function. This example is mainly concerned with comparing the accuracy of the results from both methods, particularly at the boundaries. Assuming the function created in the neural network to be $f(x, y)$, the following losses are created and minimized through training,

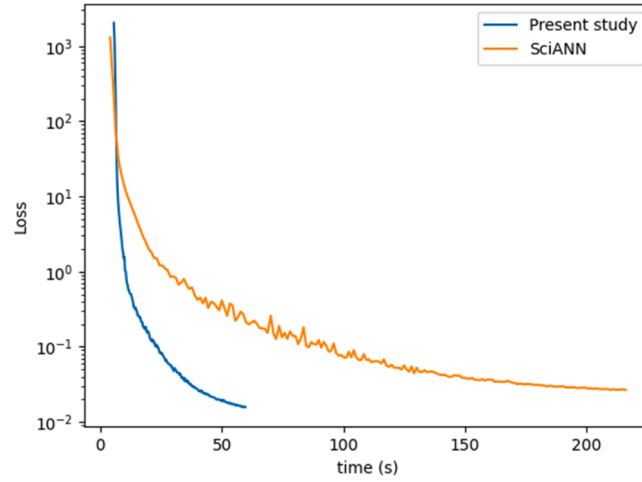


Fig. 9. Comparison of the variation of loss function versus time in the conventional PINN (SciANN) and the EBF-PINN.

$$\begin{aligned}
 R = & \sum_{\Omega} \frac{(\nabla^2 f(x_i, y_i) + k^2 f(x_i, y_i) - q(x_i, y_i))^2}{N_{\Omega}} \\
 & + \sum_{\Gamma_l} \frac{(f(-1, y_i) - \sin(-\pi) \times \sin(4\pi y_i))^2}{N_{\Gamma_l}} \\
 & + \sum_{\Gamma_r} \frac{(f(1, y_i) - \sin(\pi) \times \sin(4\pi y_i))^2}{N_{\Gamma_r}} \\
 & + \sum_{\Gamma_b} \frac{(f(x_i, -1) - \sin(\pi x_i) \times \sin(-4\pi))^2}{N_{\Gamma_b}} \\
 & + \sum_{\Gamma_t} \frac{(f(x_i, 1) - \sin(\pi x_i) \times \sin(4\pi))^2}{N_{\Gamma_t}}
 \end{aligned} \quad (7)$$

This loss is formulated using the MSE to calculate the error. The indices l, r, b, and t denote the left, right, bottom, and top boundaries, respectively. The terms N_{Ω} , N_{Γ_l} , N_{Γ_r} , N_{Γ_b} and N_{Γ_t} represent the number of nodes within the domain, and on the left, right, bottom, and top boundaries, respectively. Note that in conventional PINN, the inputs related to domain and boundaries are arranged in a series arrangement. However, in the present study, the boundary nodes and the domain nodes are imported in parallel, which optimizes calculation costs, time consumption, and convergence. It is considered that the learning rate for PINN method starts at 10^{-3} and decays exponentially to 10^{-4} after 1000 epochs. For EBF-PINN, 100 exponential bases are used and the learning rate starts from 10^{-2} and exponentially decays to 5×10^{-3} . A total of 20,000 nodes, some inside the domain and some on the boundaries, have been selected in both conventional PINN and EBF-PINN. In the Figs. 3 and 4 the results obtained using the PINN and EBF-PINN have been compared to the exact solution.

The results of EBF-PINN are more accurate and closer to the exact solution than those of PINN, as shown in Figs. 3 and 4. Also, PINN suffers from a lack of accuracy near boundaries [49], whereas EBF-PINN demonstrates more precision in the whole domain and boundaries. It should be noted that the average time for each epoch in EBF-PINN is 0.3 s, while in the PINN with the aforementioned configuration, each epoch takes 0.9 s. This represents the benefit of the presented method in terms of computation time. It should be noted that these calculations were performed on a personal computer equipped with an Intel Core i7–6700k CPU and 16 GB of RAM, without GPU acceleration.

3.2. Laplace forward example

The Laplace equation is expressed as follows:

$$\nabla^2 u(x, y) = 0, \quad (x, y) \in \Omega = (-1, 1)^2 \quad (8)$$

The solution of the Laplace equation may contain both real and imaginary parts. In order to examine the ability of the proposed method to recover complex valued solution, the exact solution has been considered in the following manner.

$$u(x, y) = F(y + i x) + G(y - i x) \quad (9)$$

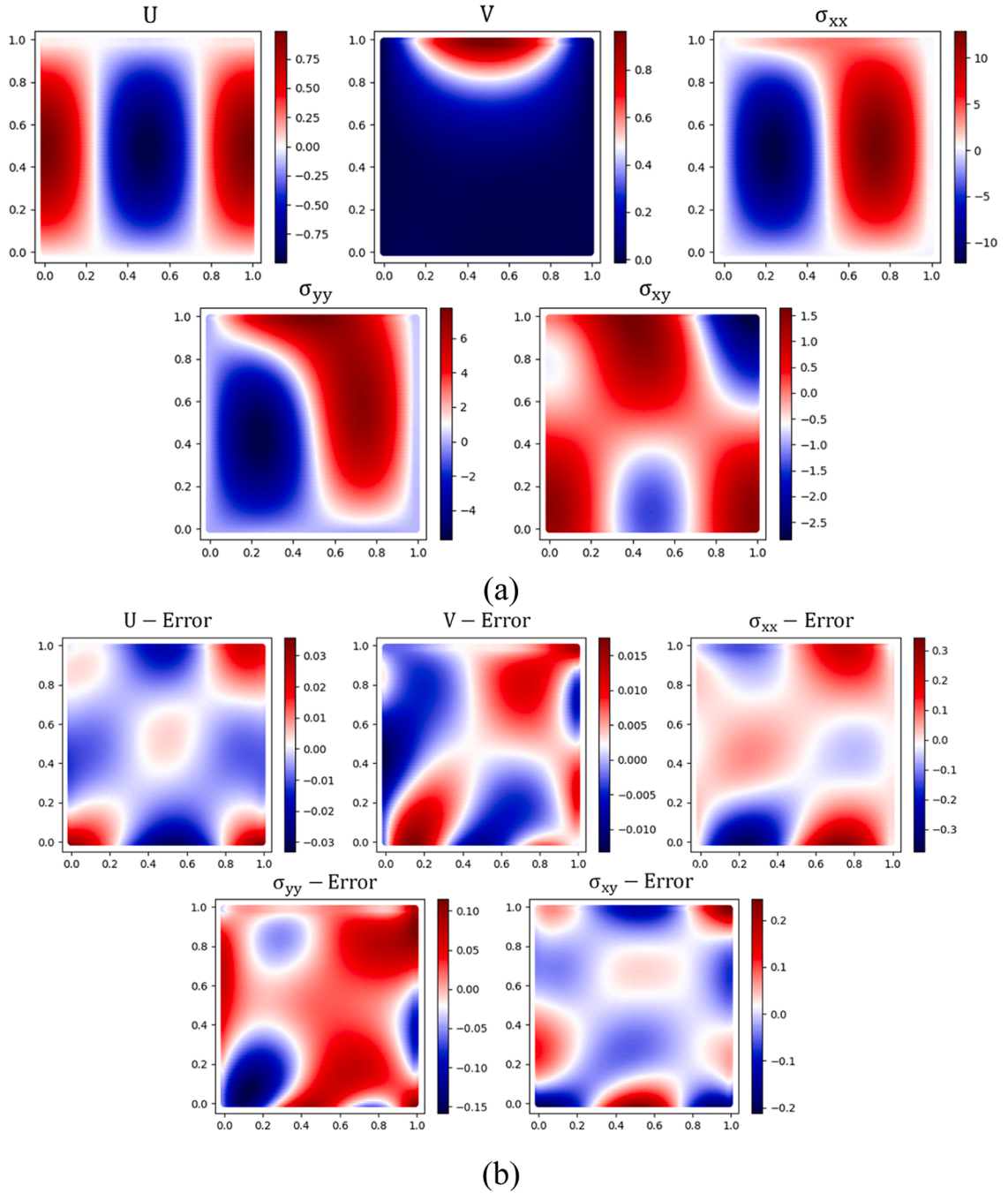


Fig. 10. Comparison of solutions of the elasto-static plane-strain investigated in Section 3.3 (a) EBF-PINN results, (b) EBF-PINN error with respect to exact solution, (c) Conventional PINN results, and (d) Conventional PINN error with respect to exact solution.

In Eq. (9), F and G are any arbitrary holomorphic functions. Considering $F(s) = 0.5s^5 - 0.2s^4 + 0.3s^2 + 0.2s$ and $G(s) = 0$, the exact solution is obtained as follows:

$$u(x, y) = 0.5(y + ix)^5 - 0.2(y + ix)^4 + 0.3(y + ix)^2 + 0.2(y + ix) \quad (10)$$

It is assumed that the boundary conditions are Dirichlet, specified along the boundaries of the square. Therefore, the boundary conditions can be extracted using the exact solution as $u(-1, y)$, $u(1, y)$, $u(x, -1)$ and $u(x, 1)$.

This problem is solved by considering 2500 nodes in the domain as well as the boundary. Three different sets of bases, i.e. 20, 50,

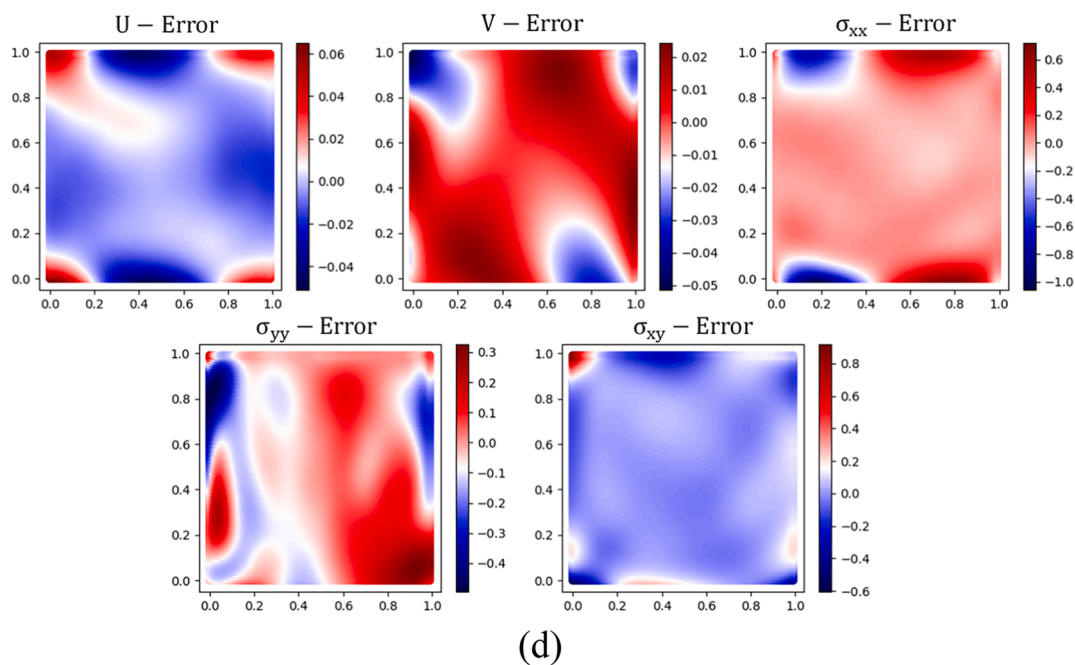
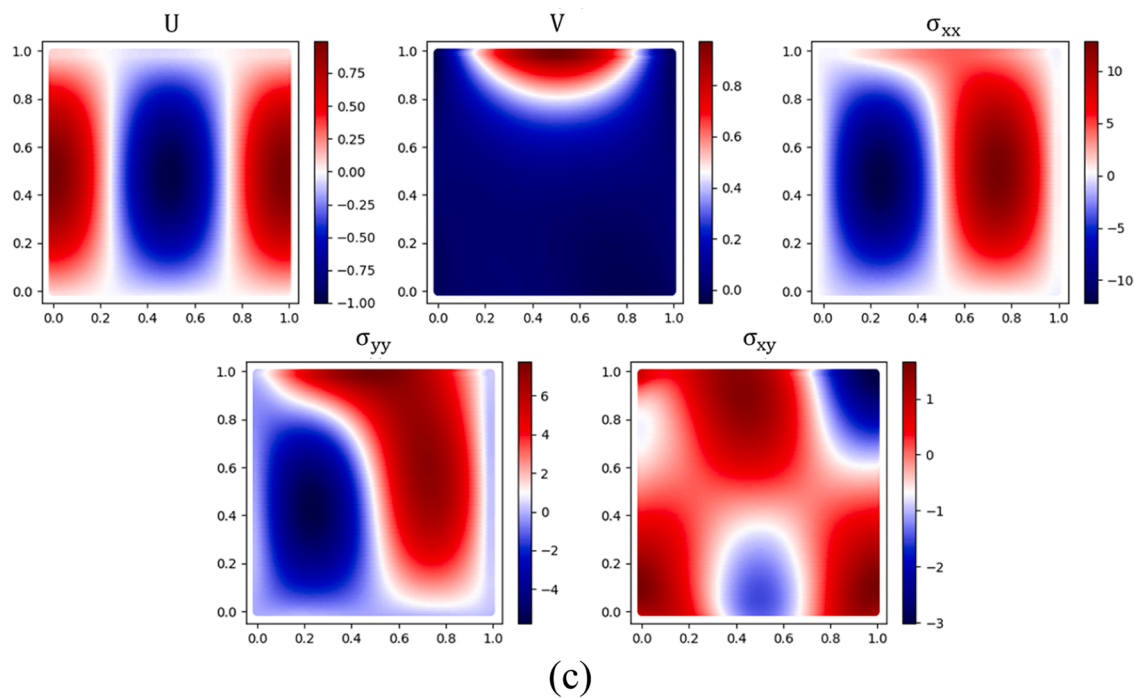


Fig. 10. (continued).

Table 1

MSE of EBF-PINN and PINN for elasto-static plane-strain forward example.

Method	U	V	σ_{xx}	σ_{yy}	σ_{xy}
EBF-PINN	6.92×10^{-5}	3.23×10^{-5}	8.20×10^{-3}	2.45×10^{-2}	2.35×10^{-3}
PINN	1.55×10^{-4}	1.72×10^{-4}	2.72×10^{-2}	1.51×10^{-2}	4.87×10^{-3}

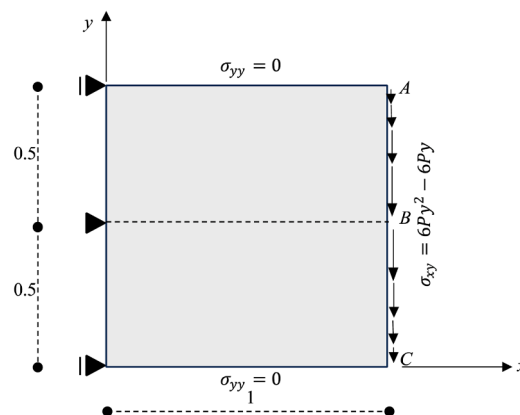


Fig. 11. The geometry and boundary conditions of the elasto-static plane-strain cantilever beam investigated in Section 3.4.

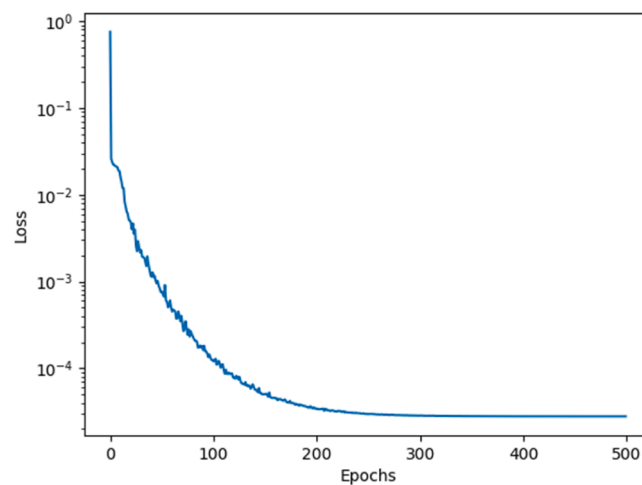


Fig. 12. Variations of loss function versus epoch number for linear analysis of cantilever beam by EBF-PINN.

Table 2

Comparison of linear analysis results obtained by FEM, EBF-PINN, PINN, and exact solution.

Node	FEM	Exact	EBF-PINN	PINN
A	-0.2638	-0.26	-0.2540	-0.2547
B	-0.2638	-0.26	-0.2534	-0.2545
C	-0.2638	-0.26	-0.2537	-0.2546

and 100 EBFs, are used for 20 trainings, and the results are compared. The initial learning rate is 0.01 and the final learning rate with exponential decay is 0.0001, which occurs in 100 epochs. Fig. 5 shows the variation of the loss function versus the epoch's number.

As Fig. 5 shows, good convergence is achieved in the solution of the Laplace equation. However, changing the number of coefficients does not affect the model's convergence rate and precision. The cause of this phenomenon can be found in the weight values. Fig. 6 shows the variation of weight values in the case of using 100 EBFs in sorted order. As shown in this Figure, the majority of the weights are located in the middle of the diagram in a linear or constant form. This indicates that depending on the problem being investigated, a higher number of weights might be unnecessary due to their effects on the results.

Fig. 7 depicts the numerical and exact solution separately for the real and imaginary parts, to enable comparison of the results of the present study with the exact solution. According to Fig. 7, EBF-PINN results in both real and imaginary domains match the exact solution with high accuracy. It is worth mentioning that the average elapsed time of each epoch using 20 EBFs is 0.3 s. The total run time for this problem is 31.68 ss. The implementation of our approach in solution of this problem can be found on GitHub [42]. This repository includes all the necessary code and instructions for reproducing the results presented here.

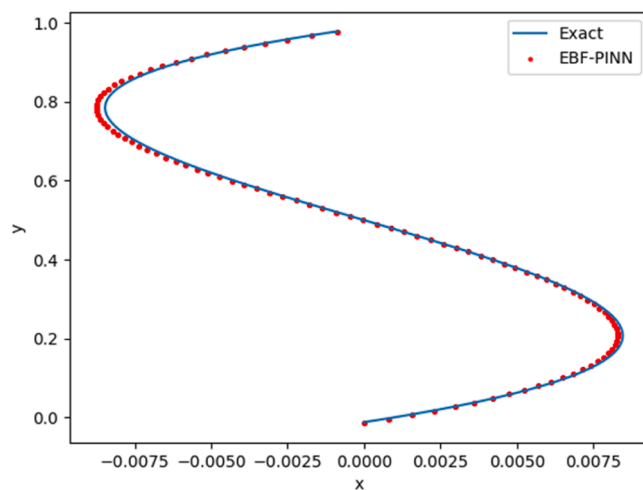


Fig. 13. Comparison of the variation of horizontal displacement on the left boundary, obtained by EBF-PINN and exact solution.

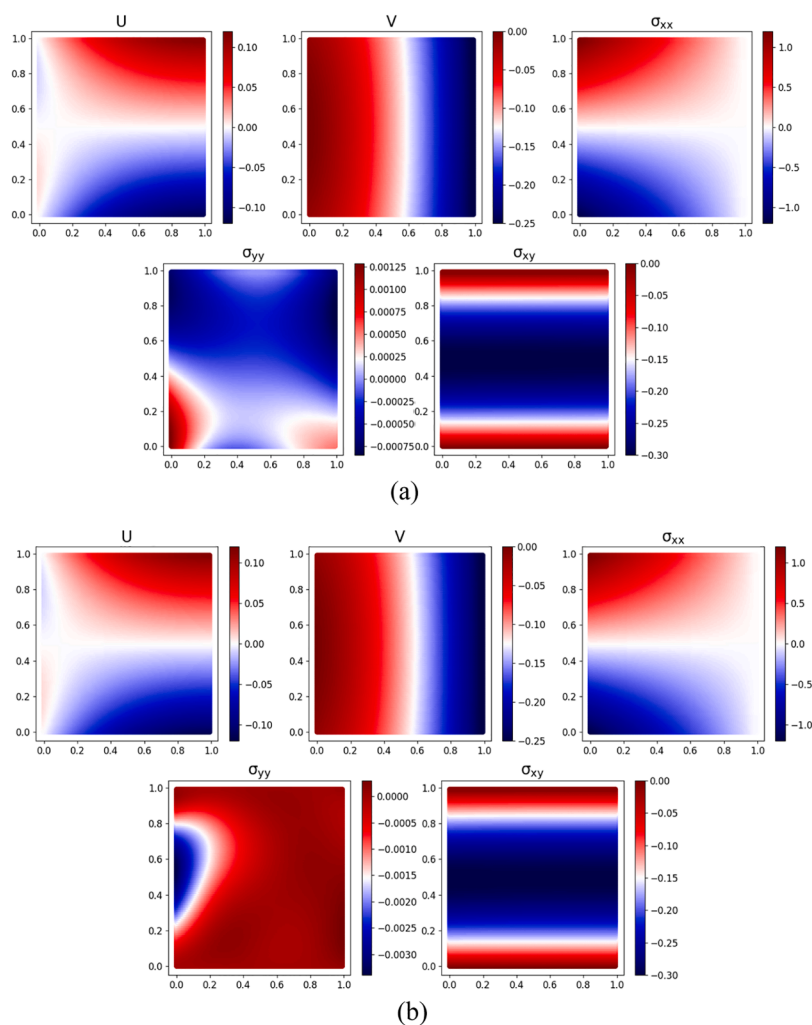


Fig. 14. Variations of displacement and stress components due to the linear analysis of the cantilever beam obtained by the (a) EBF-PINN, (b) PINN, (c) exact solution, (d) EBF-PINN error, and (e) PINN error.

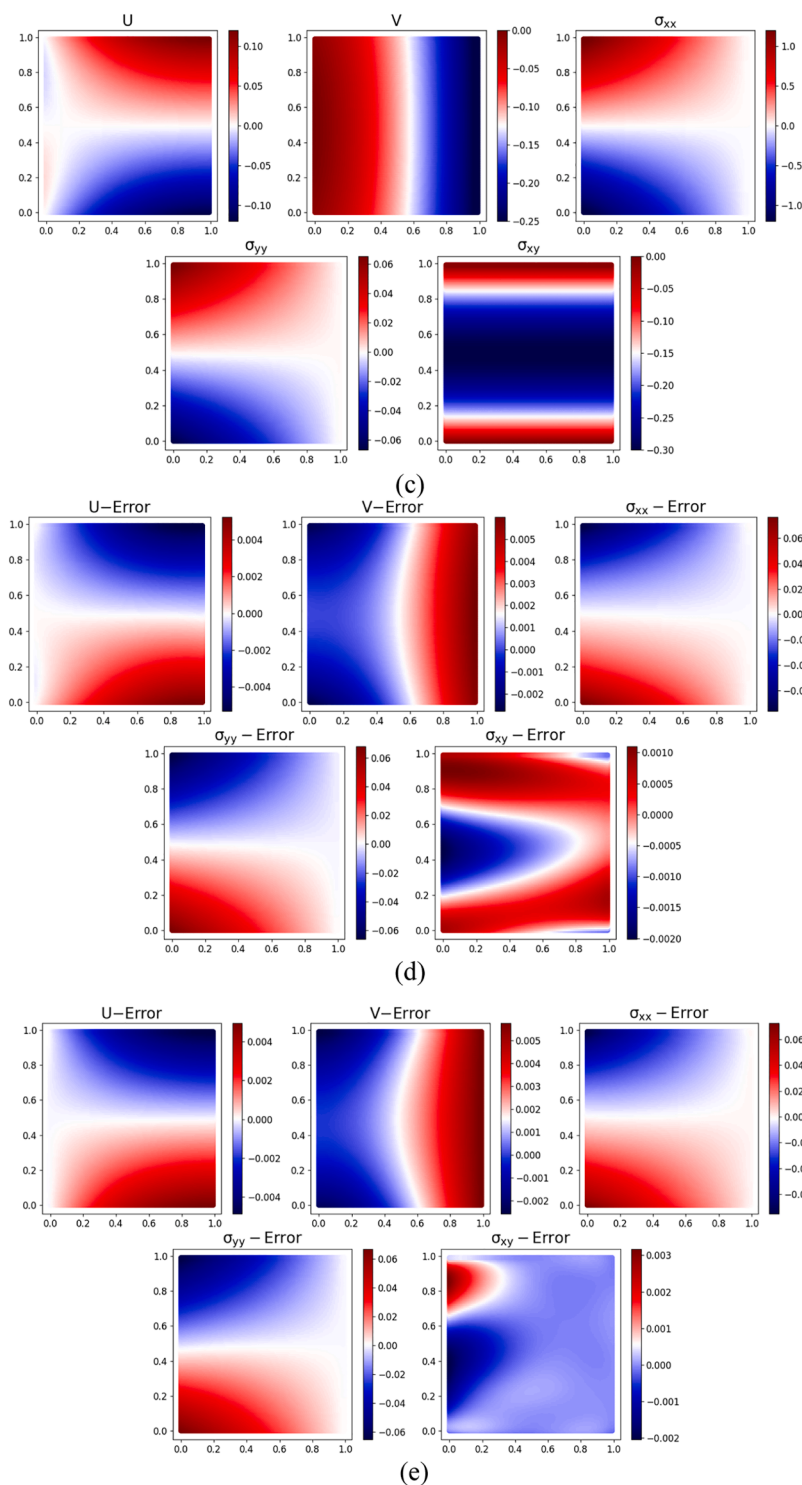


Fig. 14. (continued).

3.3. Elasto-static plane-strain forward example

The other forward example is devoted to the plane-strain problem which is introduced in [23]. Considering a plane-strain problem on a unit square subject to the boundary conditions depicted in Fig. 8. The body forces are considered as follows,

Table 3

Comparison of MSE of linear elasticity analysis of EBF-PINN and conventional PINN relative to the exact solution.

Field	EBF-PINN	PINN
U	4.54×10^{-6}	4.10×10^{-6}
V	7.04×10^{-6}	7.10×10^{-6}
σ_{xx}	5.36×10^{-4}	5.01×10^{-4}
σ_{yy}	5.13×10^{-4}	5.08×10^{-4}
σ_{xy}	4.91×10^{-7}	5.50×10^{-7}

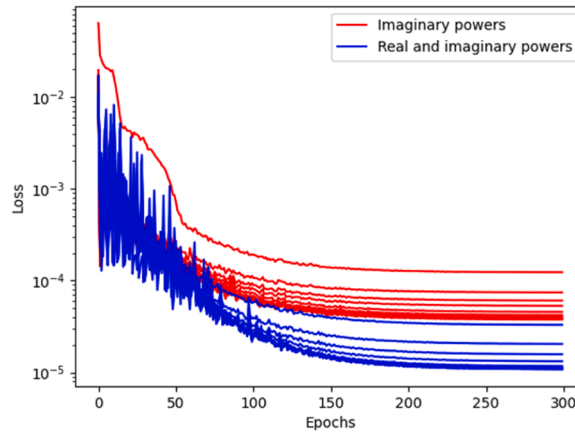


Fig. 15. Variations of loss function versus epoch number for nonlinear analysis of cantilever beam by EBF-PINN for two cases of using complex and imaginary exponents.

Table 4

Comparison of nonlinear analysis results obtained by the FEM, EBF-PINN, and PINN.

Node	FEM	EBF-PINN	PINN
A	-0.240	-0.2475	-0.2643
B	-0.234	-0.2274	-0.2418
C	-0.243	-0.2124	-0.2242

$$\begin{aligned}
 q_x(x, y) = & -\lambda [4\pi^2 \cos(2\pi x) \sin(\pi y) - \pi \cos(\pi x) Q y^3] \\
 & -\mu [\pi^2 \cos(2\pi x) \sin(\pi y) - \pi \cos(\pi x) Q y^3] \\
 & -8\mu \pi^2 \cos(2\pi x) \sin(\pi x)
 \end{aligned} \quad (11)$$

$$\begin{aligned}
 q_y(x, y) = & \lambda [3\sin(\pi x) Q y^2 - 2\pi^2 \sin(2\pi x) \cos(\pi y)] \\
 & -\mu \left[2\pi^2 \sin(2\pi x) \cos(\pi y) + \frac{Q y^4 \pi^2 \sin(\pi x)}{4} \right] \\
 & +6\mu \sin(\pi x) Q y^2
 \end{aligned} \quad (12)$$

The elasticity equations used for the calculation of losses in the domain are formulated as,

$$\sigma_{ji,j} + q_i = 0 \quad (13)$$

$$\sigma_{ij} = \lambda \delta_{ij} \varepsilon_{kk} + 2\mu \varepsilon_{ij} \quad (14)$$

$$\varepsilon_{ij} = \frac{1}{2} (u_{i,j} + u_{j,i}) \quad (15)$$

in which, σ_{ij} and ε_{ij} are the components of stress and strain tensors, respectively; δ_{ij} is Kroneker's delta; λ and μ are the Lamé constant and shear modulus of the medium, respectively. The exact solution for this example is,

$$u_x(x, y) = \cos(2\pi x) \sin(\pi y) \quad (16)$$

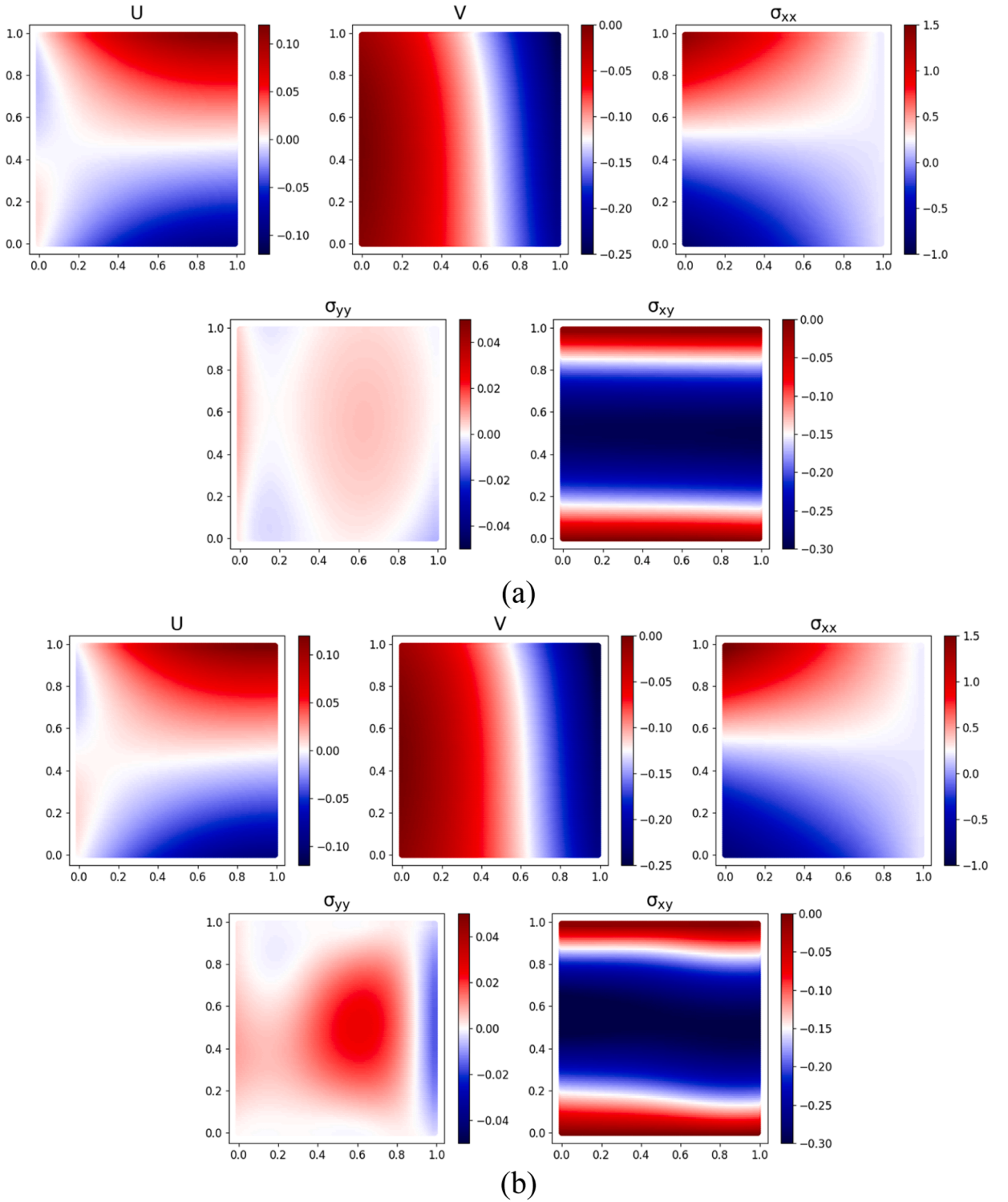


Fig. 16. Variations of displacement and stress components due to nonlinear analysis of cantilever beam obtained by (a) EBF-PINN, and (b) PINN

$$u_y(x, y) = \frac{\sin(\pi x) Q y^4}{4} \quad (17)$$

The quantities of interest in this example are u_x , u_y , σ_{xx} , σ_{yy} , σ_{xy} , ϵ_{xx} and ϵ_{xy} . As in Eqs. (13) to (15), the functions to be approximated

Table 5
Comparison of MSE of nonlinear analysis results
between EBF-PINN and PINN.

Field	MSE
U	4.89×10^{-6}
V	1.74×10^{-5}
σ_{xx}	6.23×10^{-4}
σ_{yy}	6.61×10^{-5}
σ_{xy}	1.63×10^{-4}

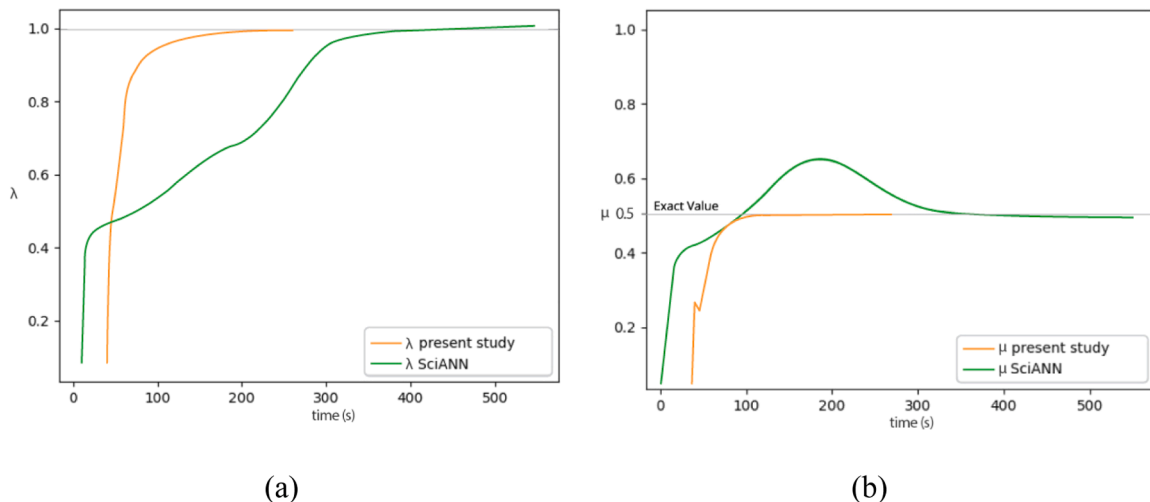


Fig. 17. Comparison of using adaptive weights for the PINN and initially trained displacement functions of EBF-PINN in the determination of (a) λ and (b) μ .

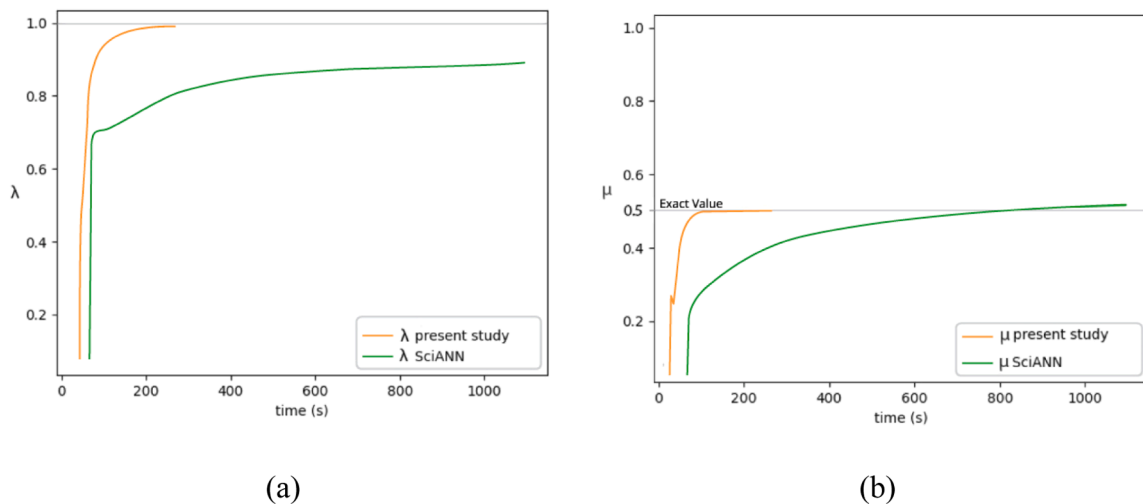


Fig. 18. Comparison of using initially trained displacement functions for both PINN and EBF-PINN methods in the determination of (a) λ and (b) μ .

in the neural network are u_x , u_y , σ_{xx} , σ_{yy} and σ_{xy} . Note that the relationship between u_i and ε_{ij} is exactly implemented in the model and constitutive equations between ε_{ij} and σ_{ij} are satisfied by the Eq. (14). The model loss is calculated as in Eq. (18) using MSE method.

$$\begin{aligned}
R = & \sum_{\Omega} \frac{(\sigma_{xx,x} + \sigma_{xy,y} + q_x(x_i, y_i))^2 + (\sigma_{xy,x} + \sigma_{yy,y} + q_y(x_i, y_i))^2}{N_{\Omega}} \\
& + \sum_{\Omega} \frac{(\sigma_{xx} - (\lambda \varepsilon_{yy} + (2\mu + \lambda) \varepsilon_{xx}))^2 + (\sigma_{yy} - (\lambda \varepsilon_{xx} + (2\mu + \lambda) \varepsilon_{yy}))^2}{N_{\Omega}} \\
& + \sum_{\Omega} \frac{(\sigma_{xy} - 2\mu \varepsilon_{xy})^2}{N_{\Omega}} + \sum_{\Gamma_b} \frac{(u_x - 0)^2 + (u_y - 0)^2}{N_{\Gamma_b}} \\
& + \sum_{\Gamma_r} \frac{(u_y - 0)^2 + (\sigma_{xx} - 0)^2}{N_{\Gamma_r}} + \sum_{\Gamma_l} \frac{(u_y - 0)^2 + (\sigma_{xx} - 0)^2}{N_{\Gamma_l}} \\
& + \sum_{\Gamma_t} \frac{(u_x - 0)^2 + (\sigma_{yy} - (\lambda + 2\mu) Q \sin(\pi x_i))^2}{N_{\Gamma_t}}
\end{aligned} \tag{18}$$

in which, $\varepsilon_{ij} = \frac{1}{2}(u_{i,j} + u_{j,i})$.

In these equations, u_x , u_y , σ_{xx} , σ_{yy} and σ_{xy} are the outputs of the model in domain and boundary nodes, as defined in the sigma indexes, N_{Ω} , N_{Γ_b} , N_{Γ_r} , N_{Γ_l} and N_{Γ_t} are the number of nodes in the domain and on bottom, right, left and top boundaries, respectively.

For the PINN solution with SciANN, an independent network with five functions is created. Each function consists of 4 layers with 40 neurons. The activation function is considered to be tanh. The constitutive parameters are assumed to be 1 and 0.5 for λ and μ and the Q in force is taken 4.0 [23]. The initial and final learning rates are 10^{-3} and 10^{-5} with exponential decay. The model is trained with 200 epochs. For domain and boundaries, a total of 20,000 nodes are used to train the model. The same number of nodes is used in the EBF-PINN model. Also, 50 exponential functions are considered. Using 200 epochs with an initial learning rate of 0.1 and a final learning rate of 0.001 for EBF-PINN with exponential decay, the variation of the loss over time is depicted in Fig. 9. It should be noted that in both methods, the hyperparameters are optimized so that the best results are achieved. Also, the batchsize in both methods is selected as 100.

The proposed EBF-PINN model shows a slight improvement concerning time and loss compared to the conventional PINN. EBF-PINN results compared to the exact solution are shown in Fig. 10. The error in this figure is defined as the difference between the exact solution and the results of the model.

Table 1 contains the MSEs of both EBF-PINN and PINN methods with respect to exact solution for all functional fields.

By comparing the results in Table 1, it can be observed that the proposed method achieves higher accuracy and faster execution speed (Fig. 10).

3.4. Linear and nonlinear analysis of elasto-static plane-strain cantilever beam

In this section, the elasto-static plane-strain cantilever linear and nonlinear beam is examined, and the results are compared to FEM and the exact solution. Considering the 2D plane-strain problem depicted in Fig. 11, the exact solution to the linear problem satisfying the boundary conditions and Eqs. (13) to (15) are as presented in the following equations.

$$u = \frac{P}{6EI} \left(y - \frac{h}{2} \right) [(6Lx - 3x^2) + (2 + \nu)(y^2 - hy)] \tag{19}$$

$$v = -\frac{P}{6EI} \left[(3Lx^2 - x^3) + 3\nu(L - x) \left(y - \frac{h}{2} \right)^2 + \frac{4 + 5\nu}{4} h^2 x \right] \tag{20}$$

where u and v are displacements along x and y coordinates, ν is Poisson ratio, E is elasticity modulus, L and h are the length and height of the beam and $I = h^3/12$. Here in this example, we consider $E = 5$, $\nu = 0.2$, $h = 1$, $L = 1$ and $P = 0.2$.

The boundary conditions are specified in Fig. 11, in which the right-side traction is parabolic shear. Interestingly, EBF-PINN imposes boundary conditions exactly at nodal boundaries. No extra conditions are used to achieve the final results. The variation of loss function versus epoch number is depicted in Fig. 12 for linear analysis.

A good convergence and loss decay are observed during model training. To compare the results, in Table 2 the vertical displacements calculated with EBF-PINN, FEM, exact solution, and PINN are compared. Noting that the results for FEM are gathered assuming a complete restrained left boundary because modeling the boundary conditions in Fig. 12 does not lead to correct results in FEM. In both EBF-PINN and PINN methods, the trainings are continued until an appropriate convergence is achieved.

A reasonable agreement is observed between EBF-PINN, PINN, and the exact solution. To investigate the precision of the presented method at the boundaries, the variation of horizontal displacement on the left boundary obtained by EBF-PINN is compared with the exact one in Fig. 13.

According to Fig. 13, despite of considering nodal boundary points, results are promising in the EBF-PINN. To distinguish the differences between exact, PINN and EBF-PINN solutions, variations of displacement and stress components have been displayed in Fig. 14. Also, the difference between exact and both methods is depicted in Fig. 14. A difference can be observed between σ_{yy} which

originated from its lower value compared to the other stress components. This difference does not lead to a significant error overall.

Table 3 presents the MSE of both methods with respect to exact solution.

To emphasize the capability of the present study, a nonlinear analysis of the cantilever beam has also been carried out. It should be noted that, by embedding the EBFs in PINN, nonlinear analysis of different PDEs is as easy as linear analysis. Here, an axial compression of 0.2 is introduced to make nonlinear results more apparent. To compare the effect of the presence of real exponents, in Fig. 15 variations of loss function versus epoch number for two cases of using complex and imaginary exponents have been compared. Although this figure shows acceptable convergence with an appropriate reduction in loss for real exponents, it shows a lower loss for complex exponents. It should be noted that the real part of the EBFs' exponents plays a quasi-role as coefficients, but as each real exponents effect is dedicated to one of the inputs at each domain point, their effect seems to be substantial for better loss decay.

Table 4 shows the results of the FEM compared to EBF-PINN and PINN.

The difference between the results of the FEM and EBF-PINN comes from the difference in the boundary conditions on the left edge. (In FEM, the horizontal displacement is assumed to be zero at all points of the left edge). Variations of displacement and stress components in the nonlinear analysis of an elasto-static cantilever beam for EBF-PINN and PINN are shown in Fig. 16.

Since the exact solution to this problem is unknown, the results of the two methods are compared with each other in MSE and shown in Table 5.

3.5. Inverse identification of material properties in an elasto-static plane-strain problem

One of the most valuable applications of PINN is inverse identification problems. For this purpose, the constitutive coefficients of the problem are estimated using differential equations and nodal information.

Here, EBF-PINN is used for identifying material parameters λ and μ in plane-strain problems. Convergence in both PINN and EBF-PINN is compared over time. Uniformly distributed data with 20×20 sample points corresponding with $\lambda = 1$, $\mu = 0.5$, and $Q = 4$ is used for both methods. The accuracy and convergence of the study depend on the architecture of the network and the available data. PINN uses a 4 layers independent network with 40 neurons. For both methods, initial values for parameters are considered as 0 and $\tanh$ has been taken for the activation function. Also, 100 exponential bases are used for EBF-PINN. To specify the parameters in EBF-PINN, initially, the displacement functions are trained using nodal sample points and then the whole model with all losses is trained to identify parameters. There are two approaches to determining the parameters of PINN: first, training the displacement function, then training the entire model for identification, and second, incorporating adaptive weights [50]. The comparison of both approaches is depicted in Figs. 17 and 18.

Using adaptive weights results in better convergence and speed versus initially training the model displacement function in PINN. However, the presented method shows better convergence with lower time and calculation costs in both schemes. Therefore, it is observed that using the current model demonstrates better performance in elasticity identification problems.

4. Conclusions

In this study, the application of exponential basis functions embedded in PINN was exhibited. Physics-Informed Neural Network is a class of deep learning in which partial differential equations are incorporated in an artificial neural network. The objective function is trained to satisfy governing equations and boundary conditions. By applying these new functions, we can achieve lower calculation costs regarding time and more precise results in both inverse and forward solutions of various BVPs. Also, using imaginary components in the proposed architecture facilitates better performance of DNN models. It also added the capability of solving problems with imaginary solutions which were shown in the sample problem of the Laplace equation. As observed, the PINN model showed a lack of precision near the boundary. This demands some consideration during training, like utilization of adaptive weights. However, EBF-PINN demonstrates better performance in satisfaction of boundary conditions. Along with all these, the lower number of variables in EBF-PINN which is accompanied by explicit differentiation leads to faster training and lower calculation costs.

The integration of EBFs into the PINN framework opens up possibilities for tackling a broader spectrum of physical problems. This includes addressing nonlinear PDEs, nonlocal integro-differential equations, and equations with time-dependent terms. As a specific suggestion for further studies, it is recommended to use EBF in the Trefftz scheme which satisfies the PDE in advance and leads to lower calculation costs while training. Although this method simplifies the determination of exponent coefficients of EBFs, which was a problem with classical methods, only linear PDEs and PDEs with constant coefficients can be solved in this case.

Acknowledgements

We would like to express our sincere gratitude to Danial Amini (Grad. student, at MIT) for his invaluable assistance in providing a basic introduction to the PINN framework.

Appendix A. The parallel feeding of data in the EBF-PINN

As it was mentioned frequently in the text, in order to train EBF-PINN, parallel scheme of feeding data into the model is used. This technique leads to faster execution time. In this Appendix, a brief comparison and explanation on the two parallel and serialized approaches for input data is presented.

Fig. A1 visually represents the architecture of a model solving the Laplace equation. This figure was generated using the provided

open-source program available at [42], and illustrates a graph-based structure for a model with two distinct inputs—domain inputs and left boundary inputs—and two corresponding outputs, U and U_{bl} . By incorporating this figure, we aim to clarify the functional relation between the domain and boundary inputs and the outputs, showcasing how EBF-PINN effectively integrates these elements for solving boundary value problems.

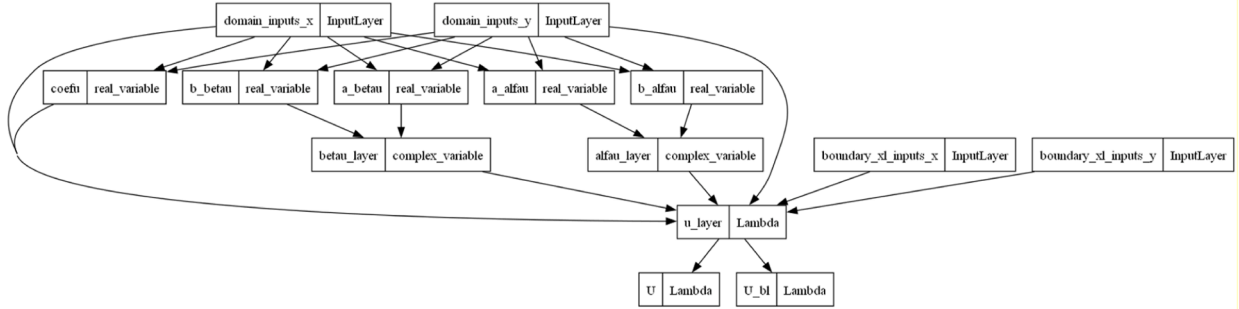


Fig. A1. A TensorFlow model with parallel input scheme used in EBF-PINN.

As illustrated in Fig. A1, the "u_layer" is first established as the core computational layer. Using this layer, the displacement fields " U " (in the domain) and " U_{bl} " (on the left boundary) are separately extracted based on the respective inputs: "domain_inputs" and "boundary_xl_inputs". This design ensures that the outputs corresponding to the domain and left boundary are generated from distinct inputs, yet they share the same trainable parameters within the "u_layer", resulting in a unified definition of " U ".

Subsequently, these displacement fields (" U " and " U_{bl} ") are utilized within their respective governing equations for the domain and the left boundary conditions. The inputs and outputs of this model are defined as,

```
inputs = [domain_inputs_x, domain_inputs_y]
inputs_bxl = [boundary_xl_inputs_x, boundary_xl_inputs_y]
outputs_list = [U, U_bl]
model = tf.keras.Model(inputs=[inputs, inputs_bxl], outputs=outputs_list)
```

In this model definition:

- Boundary nodes are passed into the model through the input variable named "inputs_bxl", which is specifically designated to handle data associated with the boundaries.
- Domain nodes are introduced into the model using the input variable named "inputs", which corresponds to the data within the domain.

This clear separation of inputs ensures that boundary and domain equations can be treated independently while still leveraging shared network parameters. By isolating boundary and domain nodes into distinct input streams, the model architecture simplifies the incorporation of boundary-specific conditions and equations into the training process.

By contrast, in the conventional PINNs approach, as depicted in Fig. A2, both " U " (domain displacement field) and " U_{bl} " (boundary displacement field) are derived simultaneously. This is achieved using a single input variable, "domain_inputs", which combines both domain and boundary nodes in a unified array. The following code shows how it is done.

```
inputs = [domain_inputs_x, domain_inputs_y]
outputs_list = [U, U_bl]
model = tf.keras.Model(inputs=[inputs], outputs=outputs_list)
```

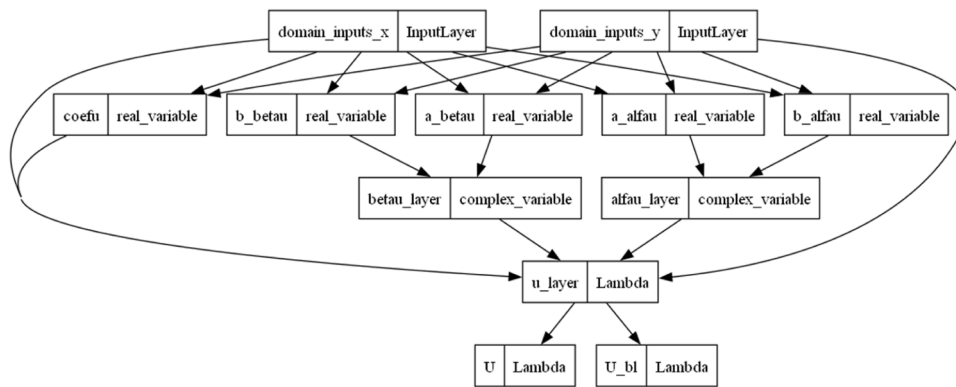


Fig. A2. A Tensorflow model with serialized input scheme used in conventional PINN.

In the serialized model definition, all nodes in the domain and on the boundary are fed into the model using the single “inputs” variable. Additionally, a sample weight list with two arrays is used to ensure that domain nodes are excluded from the boundary equation, and vice versa. The total number of input nodes is the sum of the nodes in the domain and on the boundary. As a result, the number of batches for the same batch size is effectively doubled, significantly increasing the computation time.

In contrast, the parallel input approach allows for a direct input of domain or boundary nodes, satisfying both domain and boundary equations simultaneously. The variation of loss functions versus computational time has been shown in Fig. A3. As can be distinguished, parallel feeding approach demonstrate better convergence in less time.

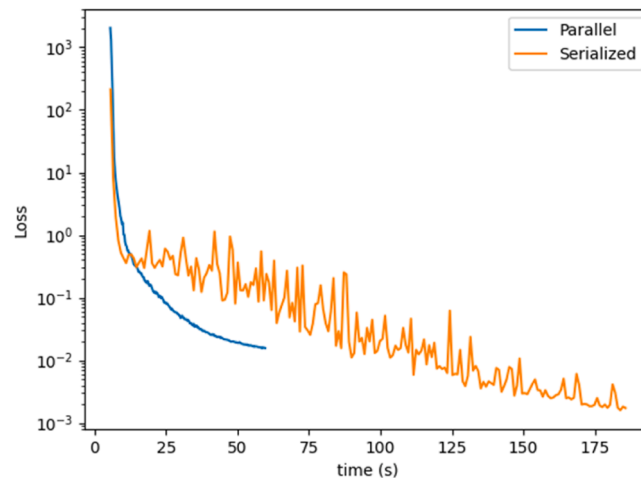


Fig. A3. A comparison of serialized and parallel input models in EBF-PINN.

Noting that the idea of designing a network based on either the parallel or serialized input approaches still requires more attention, which could be the subject of further studies.

Link to the Reproducible Capsule

The code (and data) in this article has been certified as Reproducible by Code Ocean: <https://codeocean.com/capsule/6754754/tree/v1>

Data availability

Data will be made available on request.

References

- [1] Y. LeCun, Y. Bengio, G. Hinton, Deep learning, *Nature* 521 (2015) 436–444.
- [2] C.M. Bishop, *Pattern Recognition and Machine Learning*, Springer, New York, 2016.
- [3] C.E. Yoon, O. O'Reilly, K.J. Bergen, G.C. Beroza, Earthquake detection through computationally efficient similarity search, *Sci. Adv.* 1 (2015) e1501057.
- [4] G. Pilania, C. Wang, X. Jiang, S. Rajasekaran, R. Ramprasad, Accelerating materials property predictions using machine learning, *Sci. Rep.* 3 (2013) 2810.
- [5] K.T. Butler, D.W. Davies, H. Cartwright, O. Isayev, A. Walsh, Machine learning for molecular and materials science, *Nature* 559 (2018) 547–555.

- [6] Z. Shi, E. Tsybalov, M. Dao, S. Suresh, A. Shapeev, J. Li, Deep elastic strain engineering of bandgap through machine learning, *Proc. National Acad. Sci.* 116 (2019) 4117–4122.
- [7] M.H. Rafiei, H. Adeli, A novel machine learning-based algorithm to detect damage in high-rise building structures, *Struc. Des. Tall .Special Build.* 26 (2017) e1400.
- [8] D. Sen, A. Aghazadeh, A. Mousavi, S. Nagarajiah, R. Baraniuk, A. Dabak, Data-driven semi-supervised and supervised learning algorithms for health monitoring of pipes, *Mech. Syst. Signal Process* 131 (2019) 524–537.
- [9] M. Raissi, P. Perdikaris, G.E. Karniadakis, Physics informed deep learning (part i): data-driven solutions of nonlinear partial differential equations, *arXiv preprint arXiv:1711.10561*, (2017).
- [10] J. Han, A. Jentzen, W. E, Solving high-dimensional partial differential equations using deep learning, *Proc. National Acad. Sci.* 115 (2018) 8505–8510.
- [11] Y. Bar-Sinai, S. Hoyer, J. Hickey, M.P. Brenner, Learning data-driven discretizations for partial differential equations, *Proc. National Acad. Sci.* 116 (2019) 15344–15349.
- [12] M. Raissi, P. Perdikaris, G.E. Karniadakis, Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, *J. Comput. Phys.* 378 (2019) 686–707.
- [13] S. Rudy, A. Alla, S.L. Brunton, J.N. Kutz, Data-driven identification of parametric partial differential equations, *SIAM J. Appl. Dyn. Syst.* 18 (2019) 643–660.
- [14] Y. Zhu, N. Zabarar, P.-S. Koutsourelakis, P. Perdikaris, Physics-constrained deep learning for high-dimensional surrogate modeling and uncertainty quantification without labeled data, *J. Comput. Phys.* 394 (2019) 56–81.
- [15] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G.S. Corrado, A. Davis, J. Dean, M. Devin, S. Ghemawat, I. Goodfellow, A. Harp, G. Irving, M. Isard, Y. Jia, R. Jozefowicz, L. Kaiser, M. Kudlur, J. Levenberg, D. Mané, R. Monga, S. Moore, D. Murray, C. Olah, M. Schuster, J. Shlens, B. Steiner, I. Sutskever, K. Talwar, P. Tucker, V. Vanhoucke, V. Vasudevan, F. Viégas, O. Vinyals, P. Warden, M. Wattenberg, M. Wicke, Y. Yu, X. Zheng, TensorFlow: large-scale machine learning on heterogeneous systems, 2015.
- [16] SciANN, <https://www.sciann.com/>, 2021.
- [17] E. Haghighat, R. Juanes, SciANN: a Keras/TensorFlow wrapper for scientific computations and physics-informed deep learning using artificial neural networks, *Comput. Method. Appl. Mech. Eng.* 373 (2021) 113552.
- [18] L. Lu, X. Meng, Z. Mao, G.E. Karniadakis, DeepXDE: a deep learning library for solving differential equations, *SIAM Rev.* 63 (2021) 208–228.
- [19] DeepXDE, <https://deepxde.readthedocs.io/>, 2021.
- [20] M. Raissi, G.E. Karniadakis, Hidden physics models: machine learning of nonlinear partial differential equations, *J. Comput. Phys.* 357 (2018) 125–141.
- [21] M. Raissi, A. Yazdani, G.E. Karniadakis, Hidden fluid mechanics: a Navier-Stokes informed deep learning framework for assimilating flow visualization data, *arXiv preprint arXiv:1808.04327*, (2018).
- [22] O. Kianian, S. Sarraimi, B. Movahedian, M. Azhari, PINN-based forward and inverse bending analysis of nanobeams on a three-parameter nonlinear elastic foundation including hardening and softening effect using nonlocal elasticity theory, *Eng. Comput.* (2024).
- [23] E. Haghighat, M. Raissi, A. Moure, H. Gomez, R. Juanes, A physics-informed deep learning framework for inversion and surrogate modeling in solid mechanics, *Comput. Method. Appl. Mech. Eng.* 379 (2021) 113741.
- [24] E. Haghighat, A. Can Bekar, E. Madenci, R. Juanes, Deep Learning For Solution and Inversion of Structural Mechanics and vibrations, *Modeling and Computation in Vibration Problems*, IOP Publishing, 2021. Volume 2pp. 1-1-1-17.
- [25] D. Amini, E. Haghighat, R. Juanes, Inverse modeling of nonisothermal multiphase poromechanics using physics-informed neural networks, *J. Comput. Phys.* 490 (2023) 112323.
- [26] L. Yang, D. Zhang, G.E. Karniadakis, Physics-informed generative adversarial networks for stochastic differential equations, *SIAM J. Scient. Comput.* 42 (2020) A292–A317.
- [27] B. Lusch, J.N. Kutz, S.L. Brunton, Deep learning for universal linear embeddings of nonlinear dynamics, *Nat. Commun.* 9 (2018) 4950.
- [28] T.-Y. Chang, G.-M. Gie, Y. Hong, C.-Y. Jung, Singular layer physics informed neural network method for plane parallel flows, *Comput. Mathem. Applic.* 166 (2024) 91–105.
- [29] Y. Liu, L. Cai, Y. Chen, P. Ma, Q. Zhong, Variable separated physics-informed neural networks based on adaptive weighted loss functions for blood flow model, *Comput. Mathem. Applic.* 153 (2024) 108–122.
- [30] S. Tang, X. Feng, W. Wu, H. Xu, Physics-informed neural networks combined with polynomial interpolation to solve nonlinear partial differential equations, *Comput. Mathem. Applic.* 132 (2023) 48–62.
- [31] W. Zhang, J. Li, CPINNs: a coupled physics-informed neural networks for the closed-loop geothermal system, *Comput. Mathem. Applic.* 132 (2023) 161–179.
- [32] Y. Huang, W. Hao, G. Lin, HomPINNs: homotopy physics-informed neural networks for learning multiple solutions of nonlinear elliptic differential equations, *Comput. Mathem. Applic.* 121 (2022) 62–73.
- [33] S. Cuomo, F. Giampaolo, S. Izzo, C. Nitsch, F. Piccialli, C. Trombetti, A physics-informed learning approach to Bernoulli-type free boundary problems, *Comput. Mathem. Applic.* 128 (2022) 34–43.
- [34] F.V. Difonzo, L. Lopez, S.F. Pellegrino, Physics informed neural networks for an inverse problem in peridynamic models, *Eng. Comput.* (2024).
- [35] N. Sukumar, A. Srivastava, Exact imposition of boundary conditions with distance functions in physics-informed deep neural networks, *Comput. Method. Appl. Mech. Eng.* 389 (2022) 114333.
- [36] B. Boroomand, S. Soghrati, B. Movahedian, Exponential basis functions in solution of static and time harmonic elastic problems in a meshless style, *Int. J. Numer. Method. Eng.* 81 (2010) 971–1018.
- [37] B. Shamsaei, B. Boroomand, Exponential basis functions in solution of laminated structures, *Compos. Struct.* 93 (2011) 2010–2019.
- [38] S.M. Zandi, B. Boroomand, S. Soghrati, Exponential basis functions in solution of incompressible fluid problems with moving free surfaces, *J. Comput. Phys.* 231 (2012) 505–527.
- [39] S.H. Hashemi, B. Boroomand, B. Movahedian, Exponential basis functions in space and time: a meshless method for 2D time dependent problems, *J. Comput. Phys.* 241 (2013) 526–545.
- [40] B. Movahedian, B. Boroomand, S. Soghrati, A Trefftz method in space and time using exponential basis functions: application to direct and inverse heat conduction problems, *Eng. Anal. Bound Elem.* 37 (2013) 868–883.
- [41] B. Movahedian, B. Boroomand, The solution of direct and inverse transient heat conduction problems with layered materials using exponential basis functions, *International Journal of Thermal Sciences* 77 (2014) 186–198.
- [42] S. Ghamsari, B. Movahedian, S. Sarraimi, M. Azhari, On the Use of Exponential Basis Functions in the PINN architecture: An enhanced Solution Approach For the Laplace, Helmholtz, and Elasto-Static Equations, 2025, <https://doi.org/10.24433/CO.8082916.v1>. Code Ocean.
- [43] D.P. Kingma, J. Ba, Adam: a method for stochastic optimization, *arXiv preprint arXiv:1412.6980*, (2014).
- [44] H. Robbins, S. Monro, A stochastic approximation method, *Annal. Mathem. Statist.* 22 (1951) 400–407, 408.
- [45] C. Liu, L. Zhu, M. Belkin, Loss landscapes and optimization in over-parameterized non-linear systems and neural networks, *Appl. Comput. Harmon. Anal.* 59 (2022) 85–116.
- [46] N.S. Keskar, R. Socher, Improving generalization performance by switching from adam to sgd, *arXiv preprint arXiv:1712.07628*, (2017).
- [47] D. Choi, On empirical comparisons of optimizers for deep learning, *arXiv preprint arXiv:1910.05446*, (2019).
- [48] I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning*, MIT Press, 2016.
- [49] S. Wang, Y. Teng, P. Perdikaris, Understanding and mitigating gradient flow pathologies in physics-informed neural networks, *SIAM J. Scient. Comput.* 43 (2021) A3055–A3081.
- [50] S. Wang, X. Yu, P. Perdikaris, When and why PINNs fail to train: a neural tangent kernel perspective, *J. Comput. Phys.* 449 (2022) 110768.